

Module 7 – Java – RDBMS & Database Programming with JDBC

Introduction to JDBC

• Theory:

1. What is JDBC (Java Database Connectivity)?

JDBC (Java Database Connectivity) is a standard Java API that allows Java programs to connect to and interact with databases.

In simple words:

JDBC helps Java applications send SQL queries to a database and receive results.

What JDBC is used for

Using JDBC, a Java program can:

- Connect to a database (like MySQL, Oracle, PostgreSQL)
- Execute SQL statements (SELECT, INSERT, UPDATE, DELETE)
- Retrieve data from the database
- Manage database transactions

How JDBC works (basic idea)

1. Load the JDBC driver
2. Establish a connection to the database
3. Create a statement
4. Execute SQL queries
5. Process the results
6. Close the connection

Key JDBC components

- DriverManager – manages database drivers
- Connection – represents a connection to the database
- Statement / PreparedStatement – used to execute SQL queries
- ResultSet – stores data returned from a query

o JDBC Architecture: Driver Manager, Driver, Connection, Statement, and ResultSet

2. Importance of JDBC in Java Programming ?

Importance of JDBC in Java Programming

JDBC (Java Database Connectivity) is important because it allows Java applications to store, retrieve, and manage data in databases efficiently.

Key Importance of JDBC

1. **Database Connectivity**
JDBC enables Java programs to connect with different databases such as MySQL, Oracle, and PostgreSQL.
2. **Platform Independent**
JDBC is part of Java, so it works on any platform that supports Java (Write Once, Run Anywhere).
3. **Database Independent**
The same Java code can work with different databases by changing the JDBC driver.
4. **Execution of SQL Queries**
JDBC allows execution of SQL commands like SELECT, INSERT, UPDATE, and DELETE from Java programs.
5. **Secure Data Handling**
Using PreparedStatement, JDBC helps prevent SQL injection attacks.
6. **Transaction Management**
JDBC supports transactions (commit and rollback) to maintain data consistency.
7. **Integration with Enterprise Applications**
JDBC is widely used in web applications, enterprise systems, and backend services.
8. **Efficient Data Management**
It provides a structured and efficient way to handle large amounts of data.

3. JDBC Architecture: Driver Manager, Driver, Connection, Statement, and ResultSet.

1. DriverManager

- Acts as a manager of JDBC drivers
- Loads and registers database drivers
- Establishes a connection between Java application and database

Example:

```
Connection con = DriverManager.getConnection(url, user, password);
```

2. Driver

- A software component that enables Java applications to communicate with a specific database
- Provided by database vendors (e.g., MySQL JDBC Driver, Oracle JDBC Driver)
- Converts Java JDBC calls into database-specific calls

3. Connection

- Represents a session/connection between Java program and database
- Used to create SQL statements
- Manages transactions

Example:

```
Connection con = DriverManager.getConnection(url, user, password);
```

4. Statement

- Used to execute SQL queries
- Types:
 - Statement – simple SQL queries
 - PreparedStatement – precompiled, secure queries
 - CallableStatement – used for stored procedures

Example:

```
Statement stmt = con.createStatement();
```

5. ResultSet

- Stores the result of a SELECT query
- Allows navigation through retrieved data row by row

Example:

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Flow of JDBC Architecture

Java Application



DriverManager



Driver



Database



ResultSet

2. JDBC Driver Types

- Theory: o Overview of JDBC Driver Types:

Type 1: JDBC-ODBC Bridge Driver

Type 2: Native-API Driver

Type 3: Network Protocol Driver

Type 4: Thin Driver o Comparison and Usage of Each Driver Type

Overview of JDBC Driver Types

JDBC (Java Database Connectivity) drivers enable a Java application to communicate with a database. There are **four types of JDBC drivers**, classified based on how they interact with the database.

Type 1: JDBC–ODBC Bridge Driver

Description

- Translates **JDBC calls into ODBC calls**
- Uses the **ODBC driver** installed on the client machine

Architecture

Java Application → JDBC → ODBC Bridge → ODBC Driver → Database

Features

- Platform dependent
- Requires ODBC driver installation
- Slower due to multiple translations

Advantages

- Easy to use for learning
- No database-specific driver required

Disadvantages

- Poor performance
- Not suitable for web applications
- Deprecated and removed after Java 8

Usage

- Educational purposes only
- Not used in real-world applications

Type 2: Native-API Driver

Description

- Converts JDBC calls into **native database API calls**
- Uses **database-specific client libraries**

Architecture

Java Application → JDBC → Native API → Database

Features

- Faster than Type 1
- Platform dependent
- Requires native libraries

Advantages

- Better performance than Type 1
- Direct communication with database

Disadvantages

- Not portable
- Complex deployment

Usage

- Used in controlled environments where performance is critical

Type 3: Network Protocol Driver

Description

- JDBC calls are sent to a **middleware server**
- Middleware translates them into database-specific calls

Architecture

Java Application → JDBC → Middleware Server → Database

Features

- Platform independent
- No client-side native libraries
- Supports multiple databases

Advantages

- Good for large-scale enterprise applications
- Centralized control

Disadvantages

- Requires middleware setup
- Slower than Type 4

Usage

- Enterprise applications with multiple databases

Type 4: Thin Driver (Pure Java Driver)

Description

- Converts JDBC calls **directly into database protocol**
- Written entirely in Java

Architecture

Java Application → JDBC → Database

Features

- Platform independent
- No external libraries required
- Best performance

Advantages

- Fastest and most efficient
- Easy to deploy
- Most widely used

Disadvantages

- Database-specific driver required

Usage

- Web applications
- Enterprise and cloud-based systems

3. Steps for Creating JDBC Connections

- Theory:

o Step-by-Step Process to Establish a JDBC Connection:

1. Import the JDBC packages
2. Register the JDBC driver
3. Open a connection to the database
4. Create a statement
5. Execute SQL queries
6. Process the resultset
7. Close the connection

. Import the JDBC Packages

- Required classes are present in java.sql package.

```
import java.sql.*;
```

2. Register the JDBC Driver

- Loads the database driver into memory.
- Enables Java to communicate with the database.

```
Class.forName("com.mysql.jdbc.Driver");
```

(For newer JDBC versions, this step is optional as drivers auto-register.)

3. Open a Connection to the Database

- Establishes a connection using database URL, username, and password.


```
Connection con = DriverManager.getConnection(url, user, password);
```

4. Create a Statement

- Used to send SQL queries to the database.

```
Statement stmt = con.createStatement();
```

5. Execute SQL Queries

- Executes SQL commands like SELECT, INSERT, UPDATE, DELETE.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM table_name");
```

6. Process the ResultSet

- Reads and processes the data returned by the query.

```
while(rs.next()) {  
    System.out.println(rs.getString(1));  
}
```

7. Close the Connection

- Frees database and JDBC resources.

```
rs.close();
```

```
stmt.close();
```

```
con.close();
```

4. Types of JDBC Statements

- Theory:
 - o Overview of JDBC Statements:
 - ♣ Statement: Executes simple SQL queries without parameters.
 - ♣ PreparedStatement: Precompiled SQL statements for queries with parameters.
 - ♣ CallableStatement: Used to call stored procedures
 - o Differences between Statement, PreparedStatement, and CallableStatement

1. Statement

Description

- Used to execute **simple SQL queries without parameters**
- SQL query is sent to the database **each time** it is executed

Syntax

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Features

- No parameter support
- Query is compiled every time
- Less secure (prone to SQL Injection)

Usage

- Simple, static SQL queries
- Suitable for quick testing

2. PreparedStatement

Description

- Used for **precompiled SQL queries with parameters**
- SQL is compiled **only once** and reused multiple times

Syntax

PreparedStatement ps =

```
con.prepareStatement("SELECT * FROM student WHERE id = ?");
```

```
ps.setInt(1, 101);
```

```
ResultSet rs = ps.executeQuery();
```

Features

- Supports input parameters
- Better performance
- Prevents SQL Injection

Usage

- Repeated queries
- User-input-based queries
- Most commonly used in real applications

3. CallableStatement

Description

- Used to **call stored procedures** in the database
- Can accept input and output parameters

Syntax

CallableStatement cs =

```
con.prepareCall("{call getStudent(?)}");
```

```
cs.setInt(1, 101);
```

```
ResultSet rs = cs.executeQuery();
```

Features

- Executes stored procedures
- Supports IN, OUT, and INOUT parameters
- Used for complex business logic

Usage

- Enterprise-level applications

- When logic is stored in the database

Differences Between Statement, PreparedStatement, and CallableStatement

Feature	Statement	PreparedStatement	CallableStatement
SQL Compilation	Every execution	Once (precompiled)	Precompiled
Parameters	✗ Not supported	✓ Supported	✓ Supported
Performance	Low	High	High
Security	Low (SQL Injection risk)	High	High
Stored Procedure Support	✗	✗	✓
Ease of Use	Simple	Moderate	Complex
Best Use Case	Static queries	Repeated & dynamic queries	Stored procedures

5. JDBC CRUD Operations (Insert, Update, Select, Delete)

- Theory:

- o Insert: Adding a new record to the database.

- o Update: Modifying existing records.

- o Select: Retrieving records from the database.

- o Delete: Removing records from the database.

Database Operations (CRUD)

CRUD represents the **four basic operations** performed on a database:

- Create → Insert
- Read → Select
- Update → Update
- Delete → Delete

1. Insert Operation

Definition

- **Insert** is used to **add a new record (row)** into a database table.

SQL Syntax

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

Example

```
INSERT INTO student VALUES (101, 'Rahul', 20);
```

Purpose

- Storing new data in the database
- Used during registration, data entry, form submission

2. Update Operation

Definition

- **Update** is used to **modify existing records** in a table.

SQL Syntax

UPDATE table_name

SET column1 = value1

WHERE condition;

Example

UPDATE student

SET age = 21

WHERE id = 101;

Purpose

- Correcting or changing stored data
- Updating user details, marks, or status

3. Select Operation

Definition

- **Select** is used to **retrieve records** from a database.

SQL Syntax

SELECT column1, column2 FROM table_name;

Example

SELECT * FROM student;

Purpose

- Displaying data
- Generating reports
- Searching records

4. Delete Operation

Definition

- **Delete** is used to **remove records** from a database table.

SQL Syntax

DELETE FROM table_name

WHERE condition;

Example

```
DELETE FROM student  
WHERE id = 101;
```

7. ResultSet Interface

• Theory:

What is ResultSet in JDBC?

- ResultSet is an interface in JDBC used to store and manipulate data retrieved from a database.
- It represents a table of data returned by executing a SELECT query.
- The cursor initially points before the first row.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Navigating Through ResultSet

ResultSet provides methods to move the cursor and access records.

Common Navigation Methods

Method	Description
next()	Moves cursor to the next row
previous()	Moves cursor to the previous row
first()	Moves cursor to the first row
last()	Moves cursor to the last row
beforeFirst()	Moves cursor before the first row
afterLast()	Moves cursor after the last row

Working with ResultSet to Retrieve Data

Data can be retrieved using:

- Column index (starts from 1)
- Column name

```
rs.getInt(1);
```

```
rs.getString("name");
```

Lab Exercise

Objective

To execute a SELECT query, process the ResultSet, and demonstrate cursor navigation.

Java Program: Display Records Using ResultSet

```
import java.sql.*;

public class ResultSetDemo {

    public static void main(String[] args) {

        try {

            // Load driver

            Class.forName("com.mysql.cj.jdbc.Driver");

            // Create connection

            Connection con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            // Create scrollable ResultSet

            Statement stmt = con.createStatement(

                ResultSet.TYPE_SCROLL_INSENSITIVE,

                ResultSet.CONCUR_READ_ONLY

            );

            // Execute SELECT query

            ResultSet rs = stmt.executeQuery("SELECT * FROM student");

            System.out.println("Displaying records using next():");

            while (rs.next()) {

                System.out.println(

                    rs.getInt("id") + " " +
```

```

        rs.getString("name") + " " +
        rs.getInt("age")
    );
}

// Move to first record
rs.first();
System.out.println("\nFirst Record:");
System.out.println(rs.getInt(1) + " " + rs.getString(2));

// Move to last record
rs.last();
System.out.println("\nLast Record:");
System.out.println(rs.getInt(1) + " " + rs.getString(2));

// Move to previous record
rs.previous();
System.out.println("\nPrevious Record:");
System.out.println(rs.getInt(1) + " " + rs.getString(2));

// Close resources
rs.close();
stmt.close();
con.close();

} catch (Exception e) {
    e.printStackTrace();
}
}

```

```
}
```

Sample Output

Displaying records using next():

101 Rahul 20

102 Anjali 21

103 Amit 22

First Record:

101 Rahul

Last Record:

103 Amit

Previous Record:

102 Anjali

8. Database Metadata

- Theory:
 - o What is DatabaseMetaData?
 - o Importance of Database Metadata in JDBC
 - o Methods provided by DatabaseMetaData (getDatabaseProductName, getTables, etc.)
- Lab Exercise:
 - o Write a program that retrieves and displays metadata information about your database using DatabaseMetaData.
 - o Display database name, version, list of tables, and supported SQL features.

What is DatabaseMetaData?

- **DatabaseMetaData** is an interface in JDBC that provides **information about the database itself**.
- It gives details such as:
 - o Database name and version
 - o Tables, views, schemas
 - o Supported SQL features
 - o Driver capabilities
- It is obtained from a **Connection object**.

```
DatabaseMetaData dbmd = con.getMetaData();
```

Importance of Database Metadata in JDBC

Database metadata is important because it helps:

- Understand **database structure** dynamically
- Build **database-independent applications**
- Check **database capabilities** before executing queries
- Retrieve table and column information without hardcoding

Used in:

- Database tools
- ORM frameworks

- Migration utilities
- Dynamic reporting systems

Common Methods of DatabaseMetaData

Method	Description
getDatabaseProductName()	Returns database name
getDatabaseProductVersion()	Returns database version
getDriverName()	JDBC driver name
getDriverVersion()	JDBC driver version
getTables()	Returns list of tables
supportsTransactions()	Checks transaction support
supportsBatchUpdates()	Checks batch update support
supportsStoredProcedures()	Checks stored procedure support

Lab Exercise

Objective

To retrieve and display:

- Database name
- Database version
- List of tables
- Supported SQL features

Java Program: DatabaseMetaData Example

```
import java.sql.*;
```

```
public class DatabaseMetaDataDemo {
    public static void main(String[] args) {
        try {
```

```

// Load driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Establish connection
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb",
    "root",
    "password"
);

// Get DatabaseMetaData object
DatabaseMetaData dbmd = con.getMetaData();

// Display database information
System.out.println("Database Name: " + dbmd.getDatabaseProductName());
System.out.println("Database Version: " + dbmd.getDatabaseProductVersion());
System.out.println("Driver Name: " + dbmd.getDriverName());
System.out.println("Driver Version: " + dbmd.getDriverVersion());

// Display supported features
System.out.println("\nSupported Features:");
System.out.println("Supports Transactions: " + dbmd.supportsTransactions());
System.out.println("Supports Batch Updates: " + dbmd.supportsBatchUpdates());
System.out.println("Supports Stored Procedures: " +
dbmd.supportsStoredProcedures());

// Display list of tables
System.out.println("\nList of Tables:");
ResultSet rs = dbmd.getTables(null, null, "%", new String[]{"TABLE"});
while (rs.next()) {

```

```
        System.out.println(rs.getString("TABLE_NAME"));
    }

    // Close resources
    rs.close();
    con.close();

} catch (Exception e) {
    e.printStackTrace();
}
}
```

Sample Output

Database Name: MySQL

Database Version: 8.0.35

Driver Name: MySQL Connector/J

Driver Version: 8.0.35

Supported Features:

Supports Transactions: true

Supports Batch Updates: true

Supports Stored Procedures: true

List of Tables:

student

employee

department

9. Practical SQL Query Examples

- Lab Exercise:
 - o Write SQL queries for:
 - ♣ Inserting a record into a table.
 - ♣ Updating specific fields of a record.
 - ♣ Selecting records based on certain conditions.
 - ♣ Deleting specific records.
 - o Implement these queries in Java using JDBC.

. Assumed Table Structure

We'll use a simple table called **students**:

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    age INT  
);
```

2. SQL Query Examples

a) Insert a Record

```
INSERT INTO students (id, name, email, age)  
VALUES (1, 'John Doe', 'john@example.com', 20);
```

b) Update Specific Fields

```
UPDATE students  
SET email = 'john.doe@example.com', age = 21  
WHERE id = 1;
```

c) Select Records with Conditions


```
SELECT * FROM students  
WHERE age > 18;
```

d) Delete Specific Records

```
DELETE FROM students  
WHERE id = 1;
```

3. Implementing These Queries in Java Using JDBC

a) JDBC Setup (Common Code)

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.ResultSet;  
  
public class StudentJDBC {  
    static final String URL = "jdbc:mysql://localhost:3306/school";  
    static final String USER = "root";  
    static final String PASSWORD = "password";  
  
    public static Connection getConnection() throws Exception {  
        return DriverManager.getConnection(URL, USER, PASSWORD);  
    }  
}
```

b) Insert Record (JDBC)

```
String sql = "INSERT INTO students (id, name, email, age) VALUES (?, ?, ?, ?)";  
  
try (Connection con = getConnection();  
     PreparedStatement ps = con.prepareStatement(sql)) {
```

```
ps.setInt(1, 1);  
ps.setString(2, "John Doe");  
ps.setString(3, "john@example.com");  
ps.setInt(4, 20);  
  
ps.executeUpdate();  
System.out.println("Record inserted successfully");  
}
```

c) Update Record (JDBC)

```
String sql = "UPDATE students SET email = ?, age = ? WHERE id = ?";
```

```
try (Connection con = getConnection();  
     PreparedStatement ps = con.prepareStatement(sql)) {  
  
    ps.setString(1, "john.doe@example.com");  
    ps.setInt(2, 21);  
    ps.setInt(3, 1);  
  
    ps.executeUpdate();  
    System.out.println("Record updated successfully");  
}
```

d) Select Records (JDBC)

```
String sql = "SELECT * FROM students WHERE age > ?";  
  
try (Connection con = getConnection();  
     PreparedStatement ps = con.prepareStatement(sql)) {
```

```
ps.setInt(1, 18);

ResultSet rs = ps.executeQuery();

while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name") + " " +
        rs.getString("email") + " " +
        rs.getInt("age")
    );
}
}
```

e) Delete Record (JDBC)

```
String sql = "DELETE FROM students WHERE id = ?";

try (Connection con = getConnection();
    PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setInt(1, 1);

    ps.executeUpdate();

    System.out.println("Record deleted successfully");
}
```

10. Practical

Example 1:

Swing GUI for CRUD Operations

• Theory:

o Introduction to Java Swing for GUI development o How to integrate Swing components with JDBC for CRUD operations

- **JDBC (Java Database Connectivity)** allows Java applications to interact with databases.
- Swing components capture user input.
- On button click:
 - o Input is read from text fields
 - o SQL query is executed using JDBC
 - o Output is displayed in the GUI or console

• Lab Exercise:

- o Create a simple Swing GUI with input fields for id, fname, lname, and email.
- o Implement CRUD operations (Insert, Update, Select, Delete) using JDBC and MySQL.
- o On button clicks, the program should interact with the database and perform the appropriate operation (insert, update, display records, or delete records).

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import java.sql.*;
```

```
public class CrudSwing extends JFrame {
```

```
    JTextField txtId, txtFname, txtLname, txtEmail;
```

```
    JButton btnInsert, btnUpdate, btnDelete, btnSelect;
```

```
Connection con;
```

```
public CrudSwing() {  
    setTitle("CRUD Operations using Swing & JDBC");  
    setSize(400, 350);  
    setLayout(new GridLayout(6, 2));  
    setDefaultCloseOperation(EXIT_ON_CLOSE);  
  
    JLabel lblId = new JLabel("ID:");  
    JLabel lblFname = new JLabel("First Name:");  
    JLabel lblLname = new JLabel("Last Name:");  
    JLabel lblEmail = new JLabel("Email:");  
  
    txtId = new JTextField();  
    txtFname = new JTextField();  
    txtLname = new JTextField();  
    txtEmail = new JTextField();  
  
    btnInsert = new JButton("Insert");  
    btnUpdate = new JButton("Update");  
    btnDelete = new JButton("Delete");  
    btnSelect = new JButton("Select");  
  
    add(lblId); add(txtId);  
    add(lblFname); add(txtFname);  
    add(lblLname); add(txtLname);  
    add(lblEmail); add(txtEmail);  
    add(btnInsert); add(btnUpdate);
```

```
add(btnDelete); add(btnSelect);
```

```
connect();
```

```
btnInsert.addActionListener(e -> insertRecord());
```

```
btnUpdate.addActionListener(e -> updateRecord());
```

```
btnDelete.addActionListener(e -> deleteRecord());
```

```
btnSelect.addActionListener(e -> selectRecord());
```

```
}
```

```
// Database Connection
```

```
void connect() {
```

```
    try {
```

```
        Class.forName("com.mysql.cj.jdbc.Driver");
```

```
        con = DriverManager.getConnection(
```

```
            "jdbc:mysql://localhost:3306/testdb",
```

```
            "root",
```

```
            "password"
```

```
        );
```

```
    } catch (Exception e) {
```

```
        JOptionPane.showMessageDialog(this, e.getMessage());
```

```
    }
```

```
}
```

```
// INSERT
```

```
void insertRecord() {
```

```
    try {
```

```
        String sql = "INSERT INTO users VALUES (?, ?, ?, ?)";
```

```
        PreparedStatement ps = con.prepareStatement(sql);
```

```

        ps.setInt(1, Integer.parseInt(txtId.getText()));

        ps.setString(2, txtFname.getText());
        ps.setString(3, txtLname.getText());
        ps.setString(4, txtEmail.getText());

        ps.executeUpdate();

        JOptionPane.showMessageDialog(this, "Record Inserted");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, e.getMessage());
    }
}

// UPDATE
void updateRecord() {
    try {
        String sql = "UPDATE users SET fname=?, lname=?, email=? WHERE id=?";
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setString(1, txtFname.getText());
        ps.setString(2, txtLname.getText());
        ps.setString(3, txtEmail.getText());
        ps.setInt(4, Integer.parseInt(txtId.getText()));

        ps.executeUpdate();

        JOptionPane.showMessageDialog(this, "Record Updated");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, e.getMessage());
    }
}

```

```
}
```

```
// DELETE
```

```
void deleteRecord() {
```

```
    try {
```

```
        String sql = "DELETE FROM users WHERE id=?";
```

```
        PreparedStatement ps = con.prepareStatement(sql);
```

```
        ps.setInt(1, Integer.parseInt(txtId.getText()));
```

```
        ps.executeUpdate();
```

```
        JOptionPane.showMessageDialog(this, "Record Deleted");
```

```
    } catch (Exception e) {
```

```
        JOptionPane.showMessageDialog(this, e.getMessage());
```

```
    }
```

```
}
```

```
// SELECT
```

```
void selectRecord() {
```

```
    try {
```

```
        String sql = "SELECT * FROM users WHERE id=?";
```

```
        PreparedStatement ps = con.prepareStatement(sql);
```

```
        ps.setInt(1, Integer.parseInt(txtId.getText()));
```

```
        ResultSet rs = ps.executeQuery();
```

```
        if (rs.next()) {
```

```
            txtFname.setText(rs.getString("fname"));
```

```
            txtLname.setText(rs.getString("lname"));
```



```

        txtEmail.setText(rs.getString("email"));
    } else {
        JOptionPane.showMessageDialog(this, "Record Not Found");
    }
} catch (Exception e) {
    JOptionPane.showMessageDialog(this, e.getMessage());
}
}

public static void main(String[] args) {
    new CrudSwing().setVisible(true);
}
}

```

11. Practical

Example 2: Callable Statement with IN and OUT Parameters

• Theory:

o What is a CallableStatement?

A CallableStatement is a JDBC interface used to execute stored procedures in a database.

Definition (Exam Ready)

CallableStatement is used in JDBC to call stored procedures that are stored and executed on the database server. It allows passing IN, OUT, and INOUT parameters.

Key Features

- Executes precompiled SQL stored procedures
- Improves performance
- Supports IN, OUT, and INOUT parameters
- Part of java.sql package

Syntax

```
CallableStatement cs = con.prepareCall("{call procedure_name(?, ?, ?)}");
```

o How to call stored procedures using CallableStatement in JDBC

Step-by-Step Process

1. Create a database connection
2. Write a stored procedure in MySQL
3. Prepare the CallableStatement
4. Set IN parameters
5. Register OUT parameters
6. Execute the procedure
7. Retrieve OUT parameter values

o Working with IN and OUT parameters in stored procedures

DELIMITER //

CREATE PROCEDURE insertUser(

 IN uid INT,

 IN fname VARCHAR(50),

 IN lname VARCHAR(50),

 IN email VARCHAR(100)

)

BEGIN

 INSERT INTO users VALUES (uid, fname, lname, email);

END //

DELIMITER ;

- **Lab Exercise:**

- o Create a stored procedure in MySQL with IN and OUT parameters (e.g., a procedure that takes an employee ID as input and returns the employee's full name as output).
- o Write a Java program that uses CallableStatement to call this stored procedure.
- o Demonstrate how to pass IN parameters and retrieve OUT parameters.

1. Create Stored Procedure in MySQL (IN & OUT Parameters)

1.1 Employee Table

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    fname VARCHAR(50),  
    lname VARCHAR(50)  
);
```

Sample Data

```
INSERT INTO employee VALUES (101, 'John', 'Doe');  
INSERT INTO employee VALUES (102, 'Jane', 'Smith');
```

1.2 Stored Procedure

Purpose:

- Takes **employee ID** as **IN parameter**
- Returns **full name** as **OUT parameter**

```
DELIMITER //
```

```
CREATE PROCEDURE getEmployeeFullName(  
    IN eid INT,  
    OUT fullName VARCHAR(100)  
)  
  
BEGIN  
    SELECT CONCAT(fname, ' ', lname)
```

```
    INTO fullName
  FROM employee
  WHERE emp_id = eid;
END //
```

```
DELIMITER ;
```

2. Java Program Using CallableStatement

2.1 Java Code (IN & OUT Parameters)

```
import java.sql.*;

public class CallableExample {

    public static void main(String[] args) {

        try {

            // Load MySQL Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Database Connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "password"
            );

            // Prepare CallableStatement
            CallableStatement cs =
                con.prepareCall("{call getEmployeeFullName(?, ?)}");
```

```

// Pass IN parameter
cs.setInt(1, 101);

// Register OUT parameter
cs.registerOutParameter(2, Types.VARCHAR);

// Execute stored procedure
cs.execute();

// Retrieve OUT parameter
String fullName = cs.getString(2);

System.out.println("Employee Full Name: " + fullName);

con.close();
}
catch (Exception e) {
    e.printStackTrace();
}
}
}

```

3. Explanation (Exam Friendly)

Step-by-Step Flow

1. **IN parameter** (eid) is passed using `setInt()`
2. **OUT parameter** (fullName) is registered using `registerOutParameter()`
3. Stored procedure executes on MySQL server
4. Result is retrieved using `getString()`

4. Important Methods Used

Method	Purpose
prepareCall()	Calls stored procedure
setInt()	Passes IN parameter
registerOutParameter()	Registers OUT parameter
execute()	Executes procedure
getString()	Retrieves OUT parameter

5. Output

Employee Full Name: John Doe

Lab Assignment 1: Simple JDBC Program

Write a Java program that connects to a MySQL database and executes a simple query to retrieve all records from a table

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.Statement;

import java.sql.ResultSet;


public class SelectAllRecords {


    public static void main(String[] args) {


        try {

            // Load MySQL JDBC Driver

            Class.forName("com.mysql.cj.jdbc.Driver");


            // Establish Connection

            Connection con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/testdb",

                "root",

                "password"

            );


            // Create Statement

            Statement stmt = con.createStatement();


            // SQL Query

            String sql = "SELECT * FROM employee";


            // Execute Query
```

```

        ResultSet rs = stmt.executeQuery(sql);

        // Process ResultSet
        while (rs.next()) {
            System.out.println(
                rs.getInt("emp_id") + " " +
                rs.getString("fname") + " " +
                rs.getString("lname")
            );
        }

        // Close Connection
        con.close();
    }
    catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Lab Assignment 2:

CRUD Operations using JDBC

Write a Java program that performs the following operations on a MySQL database:

- o Insert a new record.

```
import java.sql.*;
```

```
public class JdbcCrudExample {
```



```
static final String URL = "jdbc:mysql://localhost:3306/testdb";

static final String USER = "root";

static final String PASSWORD = "password";


public static void main(String[] args) {

    try {

        // Load MySQL JDBC Driver
        Class.forName("com.mysql.jdbc.Driver");

        // Create Connection
        Connection con = DriverManager.getConnection(URL, USER, PASSWORD);

        // INSERT
        String insertSQL = "INSERT INTO student VALUES (?, ?, ?, ?)";
        PreparedStatement psInsert = con.prepareStatement(insertSQL);
        psInsert.setInt(1, 1);
        psInsert.setString(2, "John");
        psInsert.setInt(3, 20);
        psInsert.setString(4, "john@example.com");
        psInsert.executeUpdate();
        System.out.println("Record Inserted");

        // UPDATE
        String updateSQL = "UPDATE student SET age=? WHERE id=?";
        PreparedStatement psUpdate = con.prepareStatement(updateSQL);
        psUpdate.setInt(1, 21);
        psUpdate.setInt(2, 1);
```

```

psUpdate.executeUpdate();

System.out.println("Record Updated");


// SELECT

String selectSQL = "SELECT * FROM student";

Statement stmt = con.createStatement();

ResultSet rs = stmt.executeQuery(selectSQL);


System.out.println("\nStudent Records:");

while (rs.next()) {

    System.out.println(

        rs.getInt("id") + " " +

        rs.getString("name") + " " +

        rs.getInt("age") + " " +

        rs.getString("email")

    );

}


// DELETE

String deleteSQL = "DELETE FROM student WHERE id=?";

PreparedStatement psDelete = con.prepareStatement(deleteSQL);

psDelete.setInt(1, 1);

psDelete.executeUpdate();

System.out.println("\nRecord Deleted");


// Close Connection

con.close();

}

catch (Exception e) {

```

```
        e.printStackTrace();  
    }  
}  
}
```

- o Update an existing record.
- o Select and display records.
- o Delete a record

Lab Assignment 3: Swing GUI with JDBC

1. Create a Swing-based GUI with fields for id, fname, lname, and email.
2. Implement buttons for Insert, Update, Select, and Delete.
3. Perform the corresponding JDBC operations for each button click.

Lab Assignment 4: Using CallableStatement

1. Create a stored procedure in MySQL with IN and OUT parameters.
2. Write a Java program that calls the stored procedure using CallableStatement and demonstrates how to pass parameters and retrieve results.